Guide: Running Playwright with a Local Browser (Single Tab Reuse Mode)

Overview

This guide explains how to run your own locally installed browser (Chrome, Edge, or Brave) with Playwright, connect using Chrome DevTools Protocol (CDP), and efficiently download hundreds of webpages by reusing one single tab.

1. Requirements

```
System:
- Ubuntu or any Linux system
- Python 3.9+
- uv or pip for package management

Install Playwright:
    uv add playwright
    # or
    pip install playwright

Then install browser drivers:
    playwright install
```

2. Launch Your Local Browser with Remote Debugging

Playwright connects to a local browser via Chrome DevTools Protocol (CDP). You must start your browser manually with a remote debugging port.

```
Find your browser:
    which google-chrome
    which google-chrome-stable
    which chromium-browser
    which brave-browser
    which microsoft-edge

Start with remote debugging (choose one):
    google-chrome-stable --remote-debugging-port=9222 --user-data-dir=/tmp/chrome-profile
    chromium-browser --remote-debugging-port=9222 --user-data-dir=/tmp/chrome-profile
    brave-browser --remote-debugging-port=9222 --user-data-dir=/tmp/brave-profile
    microsoft-edge --remote-debugging-port=9222 --user-data-dir=/tmp/edge-profile
```

Keep this browser window open — Playwright will connect to it instead of launching a new one.

3. Python Code – Reusing One Tab for 800+ URLs

```
from playwright.sync_api import sync_playwright
import time
import os

urls = [
    "https://www.google.com/search?q=mdalamin5+github",
    "https://www.wikipedia.org",
    "https://example.com",
]
```

```
with sync_playwright() as p:
    browser = p.chromium.connect_over_cdp("http://127.0.0.1:9222")
    context = browser.contexts[0] if browser.contexts else browser.new_context()
    page = context.pages[0] if context.pages else context.new_page()
    os.makedirs("html_pages", exist_ok=True)

    for i, url in enumerate(urls, start=1):
        print(f"[{i}/{len(urls)}] Visiting: {url}")
        try:
            page.goto(url, timeout=60000)
            page.wait_for_load_state("networkidle")
            time.sleep(1)

            title = page.title() or "no_title"
            safe_title = "".join(c if c.isalnum() else "_" for c in title)[:60]
            file_path = f"html_pages/{i:03d}_{safe_title}.html"

            with open(file_path, "w", encoding="utf-8") as f:
                f.write(page.content())

            print(f"■ Saved: {file_path}")
        except Exception as e:
            print(f"■ Failed: {url} -> {e}")

    print("■ All pages processed.")
    context.close()
```

4. How It Works

1. Connect to browser: Uses `p.chromium.connect_over_cdp("http://127.0.0.1:9222")` to connect to your running browser. 2. Reuse same tab: Only one `page` object is used; each URL is opened with `page.goto()`. 3. Save HTML: Each page's HTML is saved inside the `html_pages/` folder. 4. Safe filenames: Special characters are replaced to avoid OS errors. 5. Error handling: The script continues even if one URL fails.

5. Output Example

```
html_pages/
■■■ 001_mdalamin5_github_Google_Search.html
■■■ 002_Wikipedia.html
■■■ 003_example_com.html
```

6. Tips for Scaling

- Skip already-downloaded pages with `os.path.exists(file_path)`. - Reduce sleep time to speed up processing. - Run multiple scripts on different ports (9222, 9223, 9224) for parallel browsers.

7. Summary

```
| Feature | Supported |
|----------|------------|
| Reuse same browser | ■ |
| Reuse same tab | ■ |
| Save HTML files | ■ |
| Fast + lightweight | ■ |
| Works with Chrome / Brave / Edge | ■ |
```